



GENERACIÓN DE SERVICIOS EN RED

Índice

1. Servicios
2. Protocolos estándar de comunicación en red a nivel de aplicación.
3. Protocolos. HTTP y HTTPS.
4. Comunicación con un servidor FTP.

1. Servicios

- Los **servicios** son **programas auxiliares** utilizados en un sistema de computadoras para **gestionar una colección de recursos** y prestar su **funcionalidad** a los **usuarios y aplicaciones**.

Ejemplo: cuando enviamos un documento a una impresora que está formando parte de una red estamos usando un servicio de impresión. El acceso a este servicio está formado por el conjunto de operaciones que ofrece.

- Los **servicios de Internet** tienen una relación **cliente-servidor**.
- Las **aplicaciones** que **implementan TCP/IP**, se basan en un modelo **cliente/servidor**.

2. Protocolos estándar de comunicación en red a nivel de aplicación.

- Los modelos de red **OSI** y **TCP/IP** se estructuran como una sucesión de **capas** responsables de toda la actividad relacionada con la **comunicación** entre dispositivos a través de una red.
- Cada capa tiene un nivel de abstracción distinta, dependiendo de la tarea que tenga que realizar.
- Las **capas inferiores** son mucho **más técnicas y más complicadas** de programar que las superiores, ya que se encargan de detalles mucho más específicos relativos a la comunicación. Los programas de las **capas inferiores** se encuentran en los **controladores de las tarjetas de red** y en los **sistemas operativos**.

2. Protocolos estándar de comunicación en red a nivel de aplicación.

- Las **capas superiores** son las que se utilizan en la creación de las **aplicaciones** que **utilizan los usuarios**.
- La capa superior se denomina “**de aplicación**” tanto en el modelo OSI como en el modelo TCP/IP.

Capas según el modelo OSI		Capas según el modelo TCP/IP	
7	Aplicación	4	Aplicación
6	Presentación		
5	Sesión		
4	Transporte	3	Transporte
3	Red	2	Internet
2	Enlace de datos	1	Acceso a la red/Capa de Enlace
1	Física		

2. Protocolos estándar de comunicación en red a nivel de aplicación.

- La **capa de aplicación** del modelo de red TCP/IP contiene las **aplicaciones y los servicios que puede utilizar el usuario**.
- Es la capa más cercana a las personas y sobre ella se desarrollan las **aplicaciones de uso cotidiano**.
- Dentro de la capa de aplicación se ubican varios **protocolos** y cada uno tiene un cometido específico.
- Algunos son: HTTP y HTTPS , FTP, SMTP entre otros.

2. Protocolos estándar de comunicación en red a nivel de aplicación.

Tabla resumen capa-protocolo en modelo TCP/IP:

NIVELES		PROTOCOLOS	
APLICACIÓN	->	SMTP, TELNET, FTP, HTTP, POP3, NFS, SNMP, DNS, NTP, DHCP	
TRANSPORTE	->	TCP	UDP
INTERNET	->	IP	
ACCESO A LA RED	->	PROTOCOLOS DE SUBRED	

3. Protocolos. HTTP y HTTPS.

- El **Protocolo de Transferencia de Hipertexto** (***HTTP*** o *Hypertext Transfer Protocol*), se utiliza para la **transmisión de documentos a través de internet**. Es el protocolo fundamental de la web, permite la transferencia de las **peticiones y los recursos que intercambian clientes y servidores**.
- Se **basa** en el protocolo ***TCP***.
- Por defecto utiliza el **puerto 80**.

3. Protocolos. HTTP y HTTPS.

- El **Protocolo Seguro de Transferencia de Hipertexto** (***HTTPS*** o *Hypertext Transfer Protocol Secure*) es la versión segura del protocolo HTTP. Cifra toda la información intercambiada entre cliente y servidor. Por defecto utiliza el **puerto 443**.
- Uno de los aspectos técnicos que resulta necesario conocer de *HTTP* es el tipo de **peticiones** que se pueden realizar. (Peticiones=métodos).

3. Protocolos. HTTP y HTTPS.

- **Tipos de peticiones de HTTP (*método*):**
 - **GET:** Solicita la recuperación de un recurso. Una petición HTTP que utiliza este método recupera una entidad alojada en el servidor.
 - **POST:** Se utiliza para crear una entidad alojada en el servidor.
 - **PUT:** Se utiliza para crear o actualizar una entidad alojada en el servidor si ya existía previamente.
 - **DELETE:** Elimina una entidad alojada en el servidor.

3. Protocolos. HTTP y HTTPS.

- Otro elemento de HTTP que se debe conocer son los **códigos de respuesta**.
- Cuando se lanza una petición HTTP a un servidor, este genera una respuesta en forma de código.
- Están formados por **tres dígitos**, siendo el primero el que determina a nivel global el tipo de respuesta.

3. Protocolos. HTTP y HTTPS.

- **Grupo de códigos de retorno**

- 100 -199 Respuesta informativa.
- 200 -299 Éxito.
- 300 -399 Redirección.
- 400 -499 Error del cliente.
- 500 –599 Error del servidor.

- Algunos de los más frecuentes son el **200** (solicitud ha tenido **éxito**) y el **404** (el recurso solicitado **no ha sido encontrado**).

3. Protocolos. HTTP y HTTPS. Clases java

La librería donde están las clases es *java.net.**; Se encargarán de encontrar el servidor y negociar el protocolo HTTP.

URI/URL.

- **URI** (Uniform Resource Identifier): Identificador Uniforme de Recursos. Es el concepto general. Sirve para identificar algo (un recurso) de forma única, ya sea por su nombre o por su ubicación.
- **URL** (Uniform Resource Locator): Localizador Uniforme de Recursos. Es un tipo específico de URI que, además de identificar el recurso, te dice cómo llegar a él (su dirección y protocolo).

"Todas las URLs son URIs, pero no todas las URIs son URLs."

3. Protocolos. HTTP y HTTPS. Clases java

Analogía para entender diferencia entre URI y URL

Imagina que quieres contactar con un amigo llamado **Juan Pérez**.

- **URN (Solo el nombre): "Juan Pérez"**. Sabes *quién* es, pero no sabes dónde está ni cómo encontrarlo. Es un identificador, pero no sirve para ir a visitarle.
- **URL (Localizador): "Calle Mayor 10, 2ºA, Madrid, España"**. Esto identifica su casa y te dice *exactamente cómo llegar* (protocolo de transporte: ir en coche/andando a esa dirección).
- **URI (El conjunto): Engloba a los dos anteriores**. Tanto el nombre "Juan Pérez" como la dirección "Calle Mayor 10..." son formas de identificar a esa persona.

3. Protocolos. HTTP y HTTPS. Clases java

Analogía para entender diferencia entre **URI** y **URL**

- **En la Web:**
- `https://www.google.com/logo.png` es una **URL**. Te dice el protocolo (https), el servidor (google.com) y la ruta al archivo.
- `mailto:usuario@ejemplo.com` es una **URI**. Identifica un recurso de correo, pero no es una "localización" de un archivo físico al que puedas hacer un GET igual que a una web.

3. Protocolos. HTTP y HTTPS. Clases java

HttpURLConnection:

- Es la clase principal. *openConnection()* crea la instancia, pero **aún no se conecta** físicamente. Solo **prepara el objeto**.
- El *casting* a (**HttpURLConnection**) es necesario para poder usar comandos específicos de la web (como GET, POST o códigos 200/404).

3. Protocolos. HTTP y HTTPS. Clases java

Clase (Paquete)	Método	¿Para qué sirve en el ejemplo?	Relación con teoría
URI (java.net)	create(String)	Crea y valida la dirección web (ej. "http://...").	Define el recurso en la red.
URL (java.net)	toURL()	Convierte el URI en un localizador conectable.	Paso previo a la conexión.
URL	openConnection()	Prepara el objeto de conexión al recurso remoto.	Inicio del protocolo TCP/IP capa aplicación.

3. Protocolos. HTTP y HTTPS. Clases java

Clase (Paquete)	Método	¿Para qué sirve en el ejemplo?	Relación con teoría
HttpURLConnection	setRequestMethod("GET")	Indica que queremos descargar la web.	Métodos HTTP: GET, POST, PUT, DELETE.
HttpURLConnection	getResponseCode()	Envía la petición y devuelve el estado (ej. 200).	Códigos de estado: 200 (OK), 404 (Error).
HttpURLConnection	getInputStream()	Obtiene el canal de entrada para recibir los datos.	Transferencia de recursos cliente-servidor.

3. Protocolos. HTTP y HTTPS. Clases java

Clase (Paquete)	Método	¿Para qué sirve en el ejemplo?	Relación con teoría
InputStreamReader (java.io)	<i>Constructor</i>	Convierte los bytes recibidos en caracteres legibles.	Decodificación de la transmisión.
BufferedReader (java.io)	readLine()	Lee el HTML línea por línea.	Lectura del cuerpo del mensaje HTTP.
BufferedReader	close()	Cierra los flujos de datos para liberar memoria.	Gestión eficiente de recursos.

4. Comunicación con un servidor FTP

- Es una de las herramientas más útiles para el intercambio de ficheros entre ordenadores y es la forma habitual de publicación en Internet. El cliente puede subir, bajar, borrar ficheros o crear un directorio y el servidor lo ejecuta. Podemos acceder de dos formas:
 - **Acceso anónimo.** Sin que el cliente se autentifique y no tiene privilegios en el servidor. El usuario solo accede a un directorio público para descargar ficheros.
 - **Acceso autorizado.** El usuario está registrado y tiene privilegios en el servidor. El usuario tiene su directorio personal para subir, bajar, borrar ficheros o crear carpetas. Suele tener un espacio determinado.

4. Comunicación con un servidor FTP

- FTP, utiliza **dos conexiones TCP** distintas, una de control para establecer la **comunicación entre cliente y servidor** y otra de **transferencia de datos**. Cuando un cliente se **conecta** al servidor utiliza un puerto de forma aleatoria pero el servidor siempre utiliza el **puerto 21**. En la **transferencia** de datos se utiliza otro puerto para el cliente y el **servidor** utiliza el **20**.

4. Comunicación con un servidor FTP

- En Java tenemos la librería **Apache Commons Net**, proporciona una librería de componentes, para implementar los protocolos básicos de internet para el cliente. Incluye soportes para FTPK, SMTP, TELNET, TFTP,...
- Esta librería la usaremos para desarrollar la parte cliente. Para poder utilizarla tienes que descargarla e importarla en tu proyecto Java. Puedes descargarla la librería de Apache Commons Net en su web oficial [aquí](#).

4. Comunicación con un servidor FTP

◦ Para **importar la librería en eclipse** debes seguir los pasos siguientes:

1. Click con el botón derecho sobre tu proyecto Java.
2. Click Properties>>Java Build Path>>Libraries
3. Seleccionar Classpath y hacer click en Add External JARs...
4. Elegir commons-net-3.10.0.jar o la versión que se haya descargado y pulsar botón Abrir.
5. Pulsar botón Apply and Close
6. Ya puedes importar la librería en tu clase Java que necesites de ese proyecto:

```
import org.apache.commons.netftp.*;
```

4. Comunicación con un servidor FTP

- El **protocolo FTP** usa un esquema de **códigos de respuesta** donde cada uno de sus dígitos tiene un significado especial. Son números de tres dígitos en ASCII, el primer dígito indica si la respuesta es **buena, mala o incompleta**:
 - **1yz**: Respuesta preliminar positiva, el servidor inició la acción solicitada.
 - **2yz**: Respuesta de terminación positiva, el servidor terminó con éxito la acción solicitada.
 - **3yz**: Respuesta intermedia positiva, el servidor aceptó el comando, pero la acción solicitada necesita más información.
 - **4yz**: Respuesta de terminación negativa transitoria, el servidor no aceptó el comando, y la acción solicitada no ocurrió.
 - **5yz**: Respuesta de terminación negativa permanente, el servidor no aceptó el comando y la acción solicitada no ocurrió.

4.1. Comunicación con un servidor FTP.

Clase FTP , FTPClient y FTPFile

◦ Veamos algunas de los métodos de la clase **FTP**:

- ❑ ***void connect(String host)***, abre la conexión con el servidor FTP indicado en host.
- ❑ ***int getReplyCode()***, devuelve el valor entero del código de respuesta de la última respuesta FTP.
- ❑ ***String getReplyString()***, devuelve el texto completo de la respuesta del servidor FTP.

4.1. Comunicación con un servidor FTP.

Clase FTP , FTPClient y FTPFile

- En la siguiente tabla se muestran algunos metodos de la clase **FTPClient** (derivada de **FTP**) que utilizaremos para logeamos al servidor FTP, subir, bajar y eliminar ficheros, movemos de un directorio a otro, etc. Muchos de estos metodos devuelven un valor booleano, *true* si el método tuvo exito y *false* en caso contrario:

- ❑ ***void disconnect()***, cierra la conexion con el servidor FTP y restaura los parametros de conexion a los valores predeterminados.
- ❑ ***boolean login (String user, Siring passwd)***: Inicia sesión en el servidor FTP usando el nombre de usuario y la contraseña proporcionados. Devuelve *true* si se inicia la sesión con éxito, si no devuelve *false*.

4.1. Comunicación con un servidor FTP.

Clase FTP , FTPClient y FTPFile

- ❑ **String printWorkingDirectory ()**: Devuelve el nombre de ruta del directorio de trabajo actual
- ❑ **FTPFile [] listFiles()**: Obtiene una lista de ficheros del directorio actual como un array de objetos **FTPFile**.
- ❑ **FTPFile [] listFiles(String path)**: Obtiene una lista de ficheros del directorio indicado en *path*.
- ❑ **String[] listNames()**: Obtiene una lista de ficheros del directorio actual como un array de cadenas.
- ❑ **FTPFile [] listDirectories ()**: Obtiene la lista de los directorios que se encuentran en el directorio de trabajo actual

4.1. Comunicación con un servidor FTP.

Clase FTP , FTPClient y FTPFile

- ❑ **FTPFile [] listDirectories(String parent):** Obtiene la lista de los directorios que se encuentran en el directorio especificado en *parent*
- ❑ **Boolean changeWorkingDirectory (String pathname):** cambia el directorio de trabajo actual de la sesion FTP al indicado en *pathname*
- ❑ **Boolean changeToParentDirectory ():** Cambia al directorio padre del directorio de trabajo actual.
- ❑ **boolean setFileType (int fileType):** Establece el tipo de fichero a transferir: ASCII_FILE_TYPE (fichero ASCII), BINARY_FILE_TYPE (imagen binaria), etc.

4.1. Comunicación con un servidor FTP.

Clase FTP , FTPClient y FTPFile

- ❑ **boolean storeFile (String nombre, InputStream local):** Almacena un fichero en el servidor con el nombre indicado tomando como entrada el **InputStream**.
- ❑ **boolean retrieveFile (String nombre, OutputStream local):** Recupera un fichero del servidor y lo escribe en el **OutputStream** dado.
- ❑ **boolean deleteFile (String pathname):** Elimina un fichero en el servidor FTP

4.1. Comunicación con un servidor FTP.

Clase FTP , FTPClient y FTPFile

- ❑ **boolean rename (String antiguo, String nuevo):** Cambia el nombre de un fichero del servidor FTP
- ❑ **boolean removeDirectory (String pathname):** Elimina un directorio en el servidor FTP (si esta vacío).
- ❑ **boolean makeDirectory (String pathname):** Crea un nuevo subdirectorio en el servidor FTP en el directorio actual

Tienes el API de Apache para esta librería [aquí](#).

4.1. Comunicación con un servidor FTP.

Clase FTP , FTPClient y FTPFile

- Todos los métodos de comunicación con el servidor pueden lanzar ***IOException***. El servidor FTP puede optar por cerrar antes de tiempo una conexión si el cliente ha estado inactivo durante mas de un periodo de tiempo determinado (generalmente 900 segundos). La clase FTPClient detectara un cierre prematuro de la conexión con el servidor FTP y se puede producir la excepción ***FTPConnectionClosedException***.

4.1. Comunicación con un servidor FTP.

Clase FTP , FTPClient y FTPFile

- La clase **FTPFile** se utiliza para representar informacion acerca de los ficheros almacenados en un servidor FTP. Algunos metodos importantes son:

❑ **String getNameO**: Devuelve el nombre del fichero.

❑ **long getSize()**: Devuelve el tamaño del fichero en bytes.

❑ **int getType()**: Devuelve el tipo del fichero, 0 si es un fichero (FILE_TYPE), 1 un directorio (DIRECTORY_TYPE) y 2 un enlace simbólico (SYMBOLIC_LINK_TYPE)

4.1. Comunicación con un servidor FTP.

Clase FTP , FTPClient y FTPFile

- **String getUser():** Devuelve el nombre del usuario propietario del fichero
- **boolean isDirectory():** Devuelve *true* si el fichero es un directorio
- **boolean isFile():** Devuelve *true* si es un fichero
- **boolean isSymbolicLink():** Devuelve *true* si es un enlace simbolico

4.1. Comunicación con un servidor FTP.

Clase FTP , FTPClient y FTPFile

◦ Ejemplo conexión FTP(1/2):

```
public static void main(String[] args) {  
    FTPClient client = new FTPClient();  
    String sFTP = "192.168.0.170";//IP del servidor FTP  
    String sUser = "madrid";  
    String sPassword = "cmadrid";  
    try {  
        client.connect(sFTP);  
        boolean login = client.login(sUser,sPassword);  
    }  
}
```

4.1. Comunicación con un servidor FTP.

Clase FTP , FTPClient y FTPFile

- Ejemplo conexión FTP (2/2):

```
if (login) System.out.println("Conexión establecida con el  
servidor FTP");    //...Hacer FTP...  
else System.out.println("No se ha conseguido realizar la  
conexión al servidor FTP");  
    client.logout(); //Cierro sesión y desconecto  
    client.disconnect();  
} catch (IOException ioe) {  
    ioe.printStackTrace();  
}  
} //Fin main
```